

## Librerías utilizadas

### Descripción de las librerías:

**Pandas** – Es una librería para el análisis de datos en Python. Permite trabajar con estructuras de datos como los DataFrames (Que son tablas organizadas en filas y columnas). Gracias a esta librería es posible manipular, limpiar y analizar grandes volúmenes de información. Además facilita tareas como filtrar, ordenar y transformar datos, lo que la convierte en una herramienta importante en proyectos de análisis de datos. Pandas permite importar y exportar datos en varios formatos como CSV, Excel, JSON, SQL y HTML.

En este proyecto, Pandas se utilizó para gestionar y procesar conjuntos de datos de forma ordenada y clara. Permitió importar archivos, limpiar información incompleta, organizar registros, realizar cálculos estadísticos y preparar datos para su representación visual mediante Plotly. Su uso fue clave para optimizar el análisis y mejorar la precisión de los resultados.

**Pathlib** - Librería que permite trabajar con rutas de archivos y directorios de forma más clara, moderna y segura que utilizando cadenas de texto tradicionales. Es muy útil para comprobar si existe o no un dataset, evita errores al intentar cargar un archivo inexistente.

Se pueden crear rutas compatibles con distintos sistemas operativos Windows, Linux y macOS por ejemplo, sin preocuparse por barras invertidas o separadores. Pathlib ofrece ventajas como una sintaxis más clara y sencilla, compatibilidad con múltiples sistemas operativos, menor probabilidad de errores en rutas, programación orientada a objetos, gestión correcta de archivos y carpeta y facilita la automatización de procesos de lectura y escritura.

En nuestro proyecto, Pathlib fue muy importante para gestionar la ubicación del archivo CSV principal. Al principio, el programa solo funcionaba si se ejecutaba desde una carpeta concreta, ya que las rutas estaban definidas manualmente y generaban errores al cambiar de ubicación. Esto nos daba un problema importante, porque el archivo principal main o master dependía de una ruta fija para acceder al dataset.

Con Pathlib, pudimos definir rutas dinámicas y más concretas, permitiendo que el programa localizara automáticamente el archivo CSV independientemente de desde dónde se ejecutara el proyecto. De esta forma, conseguimos que el sistema pudiera acceder al dataset desde cualquier parte del proyecto sin errores, mejorando la organización y flexibilidad. Ejemplos de Pathlib y Pandas:

```

class DataFrame:
    def __init__(self):
        # Carpeta donde está este archivo .py
        base_path = Path(__file__).resolve().parent

        # Ruta al CSV
        csv_path = base_path / "../CSVs/StudentPerformanceFactors_realistic.csv"

        # Normaliza la ruta
        csv_path = csv_path.resolve()

        self.df = pd.read_csv(csv_path)

    def getdf(self):
        return self.df

```

Aquí Pandas sirve para leer y trabajar con los datos y pathlib gestiona la ruta del archivo de forma segura.

**Pytest** – Librería de testing para Python utilizada para comprobar que el código funciona correctamente mediante pruebas automatizadas. Su principal objetivo es detectar errores rápido, verificar que los resultados obtenidos sean los esperados y asegurar la estabilidad general del proyecto a medida que se desarrolla.

Esta librería es útil en proyectos de grupo como el nuestro, donde varias personas trabajan sobre el mismo código, ya que permite comprobar rápidamente que nuevas modificaciones o añadidos no rompan funcionalidades ya existentes. Gracias a Pytest, es posible mantener un control constante sobre el funcionamiento del programa, reduciendo fallos y mejorando la calidad del software.

Entre sus ventajas principales destacan su sintaxis sencilla, facilidad de uso, capacidad para automatizar pruebas, detección rápida de errores, mejor mantenimiento del código y compatibilidad con proyectos de cualquier tamaño. Además, permite organizar las pruebas de forma estructurada, haciendo más sencillo revisar, ampliar o corregir el proyecto en el futuro.

En nuestro caso, Pytest nos ha servido para verificar que distintas funciones del proyecto relacionadas con el análisis de datos funcionaran correctamente en todo momento. Esto ha sido especialmente importante para asegurar que los procesos de carga, tratamiento o transformación de datos no generasen errores inesperados.

Además de su utilidad técnica, Pytest también nos ayudó a mejorar el diseño y la presentación de los tests, ofreciendo una estructura más clara, organizada y estética. Esto facilitó la lectura de resultados, la identificación de fallos y una mejor comprensión del comportamiento del programa durante el desarrollo. Ejemplo:

```

@pytest.fixture
def main_mod():
    import main

    return main

def _run_main_with_inputs(main_mod, monkeypatch, entradas):
    secuencia = iter(entradas)
    llamadas = []

    monkeypatch.setattr("builtins.input", lambda _prompt="": next(secuencia))
    monkeypatch.setattr(main_mod.Menu, "print_options", lambda self: None)
    monkeypatch.setattr(main_mod.Menu, "get_option", lambda self, opcion:
    llamadas.append(opcion))

    main_mod.main()
    return llamadas

```

Esto finge entradas del usuario y evita interacción, también comprueba que el flujo funciona.

**Plotly** - Librería de visualización de datos para Python que permite crear gráficos interactivos, dinámicos y visualmente atractivos. A diferencia de otras librerías más antiguas, Plotly ofrece representaciones gráficas modernas que facilitan la exploración de información de una manera más intuitiva, profesional y estética.

Su principal ventaja es la interactividad ya que permite al usuario desplazarse, ampliar zonas concretas, pasar el cursor sobre puntos de datos para obtener información detallada y analizar resultados de forma mucho más precisa. Esto convierte a Plotly en una herramienta especialmente útil para representar tendencias, comparaciones, distribuciones y análisis visuales complejos.

Entre sus principales ventajas destacan la creación de gráficos de alta calidad, facilidad de uso, compatibilidad con grandes volúmenes de datos, diseño moderno, interactividad avanzada y posibilidad de exportar visualizaciones para presentaciones o informes. Además, se integra perfectamente con Pandas, lo que facilita transformar datos analizados en gráficos comprensibles.

```

# Modificamos el Data Frame principal para quedarnos con las columnas que queremos
y añadir el count
df_largo = self.df.groupby(["Hours_Studied", "Exam_Score", "Gender",
"Attendance"]).size().reset_index(name="Count")

# Creacion de la grafica
fig = px.scatter_3d(
    df_largo, # Data Frame modificado
    x="Hours_Studied", # Eje X: Horas de estudio
    y="Attendance", # Eje Y: Atencion en clase
    z="Exam_Score", # Eje Z: Notas de los examenes

```

```

title=self.titulo, # Titulo de la grafica
color="Gender", # Color diferenciado por genero
color_discrete_map=color_discrete_map, # Opciones de los colores
size="Count", # Radio segun cantidad
labels={ # Renombre de las distintas opciones
    "Hours_Studied": "Horas de Estudio",
    "Exam_Score": "Puntuación del Examen",
    "Attendance": "Asistencia a Clase",
},
template="plotly_dark", # Estilo de fondo de la Grafica
)

```

Aquí vemos como con Plotly se crea una gráfica 3D interactiva que analiza la relación entre varias variables relacionadas con nuestro dataset.